Object Oriented Programming(C++)
(CSE 1211)

Md. Mahfuzur Rahman
Lecturer,
Prime University

Course Outline

- Object Oriented Programming Concepts
- Beginning with C++
- Classes and Objects
- Inheritance
- Encapsulation
- Abstraction
- Overloading
- Polymorphism
- Exception Handling
- I/O Stream or File Processing
- Standard Template Library

Object Oriented Programming Concepts

- **POP and OOP?**
  - Procedural Oriented Programming (POP) is about writing procedures or functions that perform operations on the data, while object-oriented programming (OOP) is about creating objects that contain both data and functions.

Object-oriented programming has several advantages over procedural programming:

- OOP is faster and easier to execute
- OOP provides a clear structure for the programs
- OOP helps to keep the C++ code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug
- OOP makes it possible to create full reusable applications with less code and shorter development time

Differences Between OOP and POP

Parameter	Object Oriented Programming(OOP)	Procedure Oriented Programming(POP)
Definition	• OOP, refers to Object Oriented Programming and its deals with objects and their properties.	• POP, refers to Procedural Oriented Programming and its deals with programs and functions.
Approach	• An object-oriented program uses the Bottom-up approach.	• A procedure-oriented program uses the Top-down approach.
Access Control	• Access control is supported by the means of access modifiers. The access specifiers such as public, private and protected are used.	• No access modifiers are supported.
Data Hiding	• Data can be hidden using Encapsulation	• There is no data hiding mechanism. Data is globally accessible.
Entity Linkage	• Object functions are linked through message passing.	• Parameter passing is involved for message passing.
Polymorphism	• Method Overloading and Method overriding are used in OOP to achieve polymorphism.	• POP does not support polymorphism.
Virtual Function and Inheritance	• OOP supports inheritance and virtual functions and virtual classes via it.	• There is no concept of inheritance in POP and neither does it allow it to support the use of virtual classes or virtual functions
Code Reusability	• OOP supports code reusability.	• No Code Reusability is provided by POP.
Operator Overloading	• It is allowed in OOP	• Operator overloading is not allowed in POP.

List of Object Oriented Programming Languages

- C++
- C#
- Dart
- Java
- JavaScript
- Objective-C
- PHP
- Perl
- Python
- Ruby

Object Oriented Programming(OOP)

Object Oriented Programming (OOP) is an approach that helps a programmer to create an abstract data type and we can write a program by using various features like inheritance, encapsulation, hiding, polymorphism, etc.. The main aim of OOP is to bind together the data and the functions that operate on them so that no other part of the code can access this data except that function.

There are some basic concepts that act as the building blocks of OOPs i.e.

1.Objects

2.Classes

3.Encapsulation

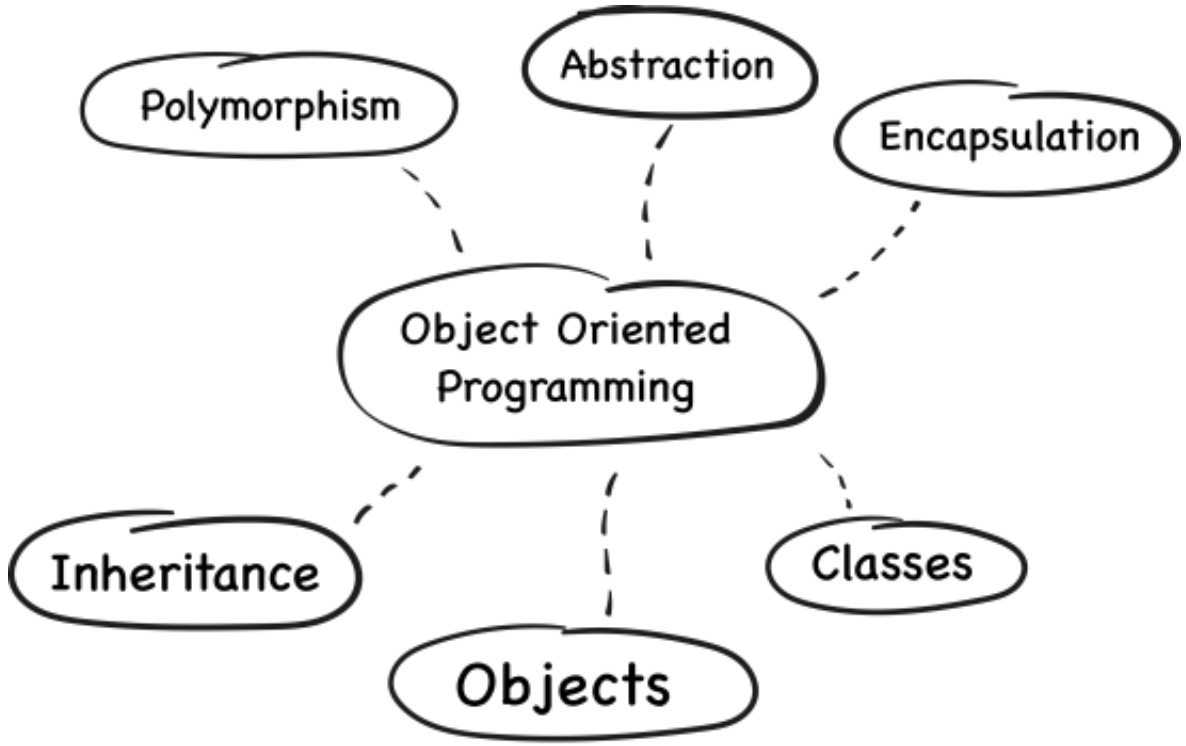
4.Abstraction

5.Polymorphism

6.Inheritance

7.Dynamic Binding

8.Message Passing

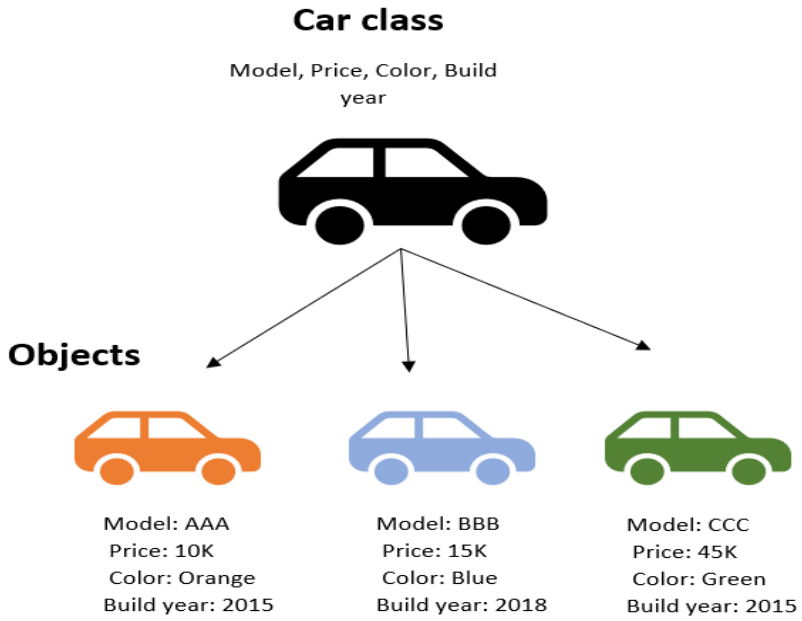


Object

An Object is an identifiable entity with some characteristics and behavior. An Object is an instance of a Class or Class type variable. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.

An object has two characteristics: **attributes** and **behavior**. For example, a car can be an object. And, it has

- **attributes** - brand, model, size, mileage, etc.
- **behavior** - driving, acceleration, parking, etc.



Classes

- A Class is a user-defined data type that has data members and member functions. It is a collection of objects of similar type.
- In the above example of class Car, the data member will be brand, model, speed limit, mileage, etc and member functions can apply brakes, increase speed, etc.

Encapsulation

In normal terms, Encapsulation is defined as wrapping up data and information under a single unit. In Object-Oriented Programming, Encapsulation is defined as binding together the data and the functions that manipulate them.

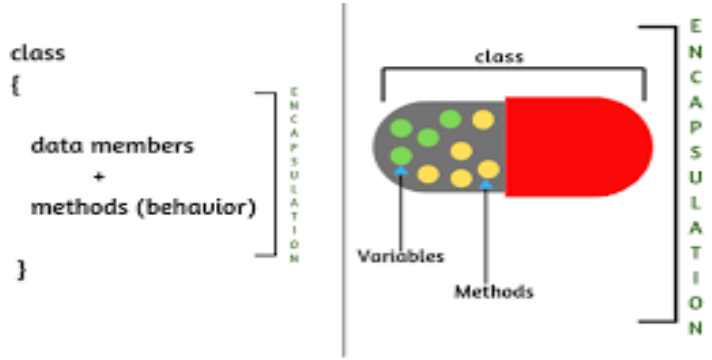


Fig: Encapsulation

Abstraction

Data abstraction is one of the most essential and important features of object-oriented programming in C++. Abstraction means displaying only essential information and hiding the details. Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation.



Real Life Example of Abstraction

Polymorphism

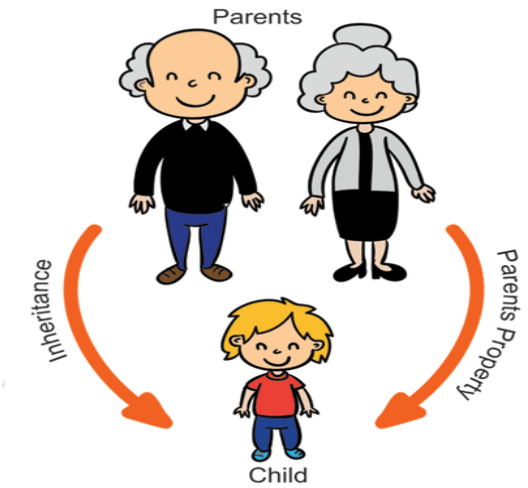
The word polymorphism means having many forms. In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form.



Sitesbay.com

Inheritance

Inheritance is the process by which objects of one class acquire the properties of another class.

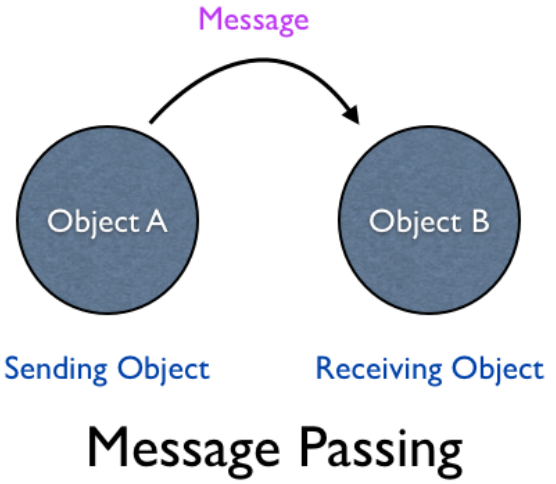


Dynamic Binding

In dynamic binding, the code to be executed in response to the function call is decided at runtime. C++ has virtual functions to support this. Because dynamic binding is flexible, it avoids the drawbacks of static binding, which connected the function call and definition at build.

Message Passing

Objects communicate with one another by sending and receiving information. A message for an object is a request for the execution of a procedure and therefore will invoke a function in the receiving object that generates the desired results. Message passing involves specifying the name of the object, the name of the function



Applications of OOP

- Real-Time Systems
- Simulation and Modeling
- Object-Oriented databases
- Hypertext, Hyper Media and Expertext
- AI and Expert Systems
- Neural Networks and Parallel Programming

C++ Programming Language

C++ history

- **C++ programming language** was developed in 1980 by Bjarne Stroustrup at bell laboratories of AT&T (American Telephone & Telegraph), located in U.S.A.
- **Bjarne Stroustrup** is known as the **founder of C++ language.**
- It was develop for adding a feature of **OOP (Object Oriented Programming)** in C without significantly changing the C component.
- C++ programming is "relative" (called a superset) of C, it means any valid C program is also a valid C++ program.



What is C++?

- C++ is a general purpose, case-sensitive, free-form programming language that supports object-oriented, procedural and generic programming.
- C++ is a middle-level language, as it encapsulates both high and low level language features.
- C++ is a cross-platform language that can be used to create high-performance applications.
- C++ was developed by Bjarne Stroustrup, as an extension to the C language.
- C++ gives programmers a high level of control over system resources and memory.
- The language was updated 4 major times in 2011, 2014, 2017, and 2020 to C++11, C++14, C++17, C++20.

Why Use C++

- C++ is one of the world's most popular programming languages.
- C++ can be found in today's operating systems, Graphical User Interfaces, and embedded systems.
- C++ is an object-oriented programming language which gives a clear structure to programs and allows code to be reused, lowering development costs.
- C++ is portable and can be used to develop applications that can be adapted to multiple platforms.
- C++ is fun and easy to learn!
- As C++ is close to C, C# and Java, it makes it easy for programmers to switch to C++ or vice versa.

Difference between C and C++

C	C++
• C follows the **procedural style programming.**	• C++ is multi-paradigm. It supports both **procedural and object oriented.**
• Data is less secured in C.	• In C++, you can use modifiers for class members to make it inaccessible for outside users.
• C follows the **top-down approach.**	• C++ follows the **bottom-up approach.**
• C does not support function overloading and operator overloading.	• C++ supports function overloading and operator overloading.
• In C, you can't use functions in structure.	• In C++, you can use functions in structure.
• C does not support reference variables.	• C++ supports reference variables.
• In C, **scanf() and printf()** are mainly used for input/output.	• C++ mainly uses stream **cin and cout** to perform input and output operations.
• C programs are divided into **procedures and modules**	• C++ programs are divided into **functions and classes.**
• C does not provide the feature of namespace.	• C++ supports the feature of namespace.
• Exception handling is not easy in C. It has to perform using other functions.	• C++ provides exception handling using Try and Catch block.
• C does not support the classes, objects and inheritance.	• C++ supports classes, objects and inheritance.

C++ Standard Libraries

Standard C++ programming is divided into three important parts:

- The core library includes the data types, variables and literals, etc.
- The standard library includes the set of functions manipulating strings, files, etc.
- The Standard Template Library (STL) includes the set of methods manipulating a data structure.

C++ Features

C++ is a widely used programming language.

1. Simple
2. Abstract Data types
3. Machine Independent or Portable
4. Mid-level programming language
5. Structured programming language
6. Rich Library
7. Memory Management
8. Quicker Compilation
9. Pointers
10. Recursion
11. Extensible
12. Object-Oriented
13. Compiler based
14. Reusability
15. National Standards
16. Errors are easily detected
17. Power and Flexibility
18. Strongly typed language
19. Redefine Existing Operators
20. Modeling Real-World Problems
21. Clarity

Basic Program

```
#include <iostream>              //Preprocessor Directive
using namespace std;             //Using Directive
int main()                       /* Program starts with main  function*/
{                                // start Function body
cout << “Hello World!\n”;        // program statement and must end with semicolon
return 0;                        //statement
}                                //end function body
```

Preprocessor Directive and Header File: In the FIRST example, the preprocessor directive **#include** tells the compiler to add the source file ***iostream*** to the **first.cpp** source file before compiling.

Header File	**Function and Description**
<iostream>	It is used to define the **cout, cin** objects, which correspond to standard output stream, standard input stream and standard error stream, respectively.

The *using* directive: A C++ program can be divided into different *namespaces*. A namespace is a part of the program in which certain names are recognized; outside of the namespace they're unknown. The directive

using namespace std;

says that all the program statements that follow are within the std namespace. Various program components such as cout are declared within this namespace. If we didn't use the using directive, we would need to add the std name to many program elements. For example, in the FIRST program we'd need to say

std::cout << "Hello World!";

To avoid adding std:: dozens of times in programs we use the using directive instead.

Output Using cout

As you have seen, the statement

cout << "Hello World!\n";

The identifier cout (pronounced “C out”) is actually an *object*. It is predefined in C++ to correspond to the *standard output stream*. A *stream* is an abstraction that refers to a flow of data. The standard output stream normally flows to the screen display—although it can be redirected to other output devices.

The operator << is called the *insertion* or *put to* operator. It directs the contents of the variable on its right to the object on its left. In **first** it directs the string constant “Hello World!\n” to cout, which sends it to the display.

C++ Program Structure

- **Header file inclusion section**: This is the section where we include all required header files whose functions we are going to use in the program.
- **Namespace section**: This is the section where we use the namespace.
- **The main() section**: In this section, we write our main code. The main() function is an entry point of any C++ programming code from where the program's execution starts.

C++ Program Structure

- The C++ language defines several headers, which contain information that is either necessary or useful to your program. For this program, the header **<iostream>** is needed.
- The line **using namespace std;** tells the compiler to use the std namespace. Namespaces are a relatively recent addition to C++.
- The next line '**// main() is where program execution begins.**' is a single-line comment available in C++. Single-line comments begin with // and stop at the end of the line.
- The line **int main()** is the main function where program execution begins.
- The next line **cout << "Hello World";** causes the message "Hello World" to be displayed on the screen.
- The next line **return 0;** terminates main() function and causes it to return the value 0 to the calling process.

Standard input stream (cin)

The **cin** is a predefined object of **istream** class. It is connected with the standard input device, which is usually a keyboard. The cin is used in conjunction with stream extraction operator (>>) to read the input from a console.
Let's see the simple example of standard input stream (cin):

```
1. #include <iostream>
2. using namespace std;
3. int main( ) {
4.  int age;
5.   cout << "Enter your age: ";
6.   cin >> age;
7.   cout << "Your age is: " << age << endl;
8. }
```

Output:

```
Enter your age: 22
Your age is: 22
```

Standard end line (endl)

The **endl** is a predefined object of **ostream** class. It is used to insert a new line characters and flushes the stream.
Let's see the simple example of standard end line (endl):

```
1. #include <iostream>  or #include <bits/stdc++.h>
2. using namespace std;
3. int main( ) {
4. cout << "C++ Tutorial";
5. cout << " Javatpoint"<<endl;
6. cout << "End of line"<<endl;
7. }
```

Output:

```
C++ Tutorial Javatpoint
End of line
```

C++ Comments

Single-line Comments

Single-line comments start with two forward slashes (//).

```
// This is a comment
cout << "Hello World!";
```

C++ Multi-line Comments

Multi-line comments start with /* and ends with */.

```
/* The code below will print the words Hello World!
to the screen, and it is amazing */
cout << "Hello World!";
```

Identify the error in the following c++ program

```
#include <iostream.h>
void main()
{
int i=0;
int i= i+1;
cout << i << endl;
/*comment\* //i=i+1;
cout << i ;
}
```

Find errors, if any, in the following c++ statements

- cout >> “i+j =“ >> -(i+j);
- cout << “x=“ x;
- m = 5; //n=10; //s=m+n;
- cin >> x; >> y;
- cout << \n ”Name: “ << name;
- cout << “Enter value:”; cin>>x;
- /*Addition*/z=x+y;

C++ Variable

A variable is a name of memory location. It is used to store data. Its value can be changed and it can be reused many times. It is a way to represent memory location through symbol so that it can be easily identified.

Let's see the syntax to declare a variable:

type variable_list;

The example of declaring variable is given below:

```
1. int x;
2. float y;
3. char z;
```

Here, x, y, z are variables and int, float, char are data types.

We can also provide values while declaring the variables as given below:

```
1. int x=5,b=10;  //declaring 2 variable of integer type
2. float f=30.8;
3. char c='A';
```

Rules for defining variables

- A variable can have alphabets, digits and underscore.
- A variable name can start with alphabet and underscore only. It can't start with digit.
- No white space is allowed within variable name.
- A variable name must not be any reserved word or keyword e.g. char, float etc.

Valid variable names:

```
1. int a;
2. int_ab;
3. int a30;
```

Invalid variable names:

```
1. int 4;
2. int x y;
3. int double;
```

Variable Declaration and Initialization

- In c++, a variable can be declared anywhere in the program. But in c, variable must be declared first after the main function.

```
void main(void)
{
int p;
p=10;
int q;      // No error in c++ but in c, compiler will generate error.
…….
}
```

- **Variable Initialization:** Both c and c++ initialize the variable using assignment operator (=).

```
int a=20;
float b=3.4;
```

But in c++, variable also initialized by base type constructor instead of assignment operator. **The syntax is:** ***data_type variable_name(value);*** For example,

```
int a(20);
float b(3.4);
```

Variable

```
int myNum;
myNum = 15;
cout << myNum;
```

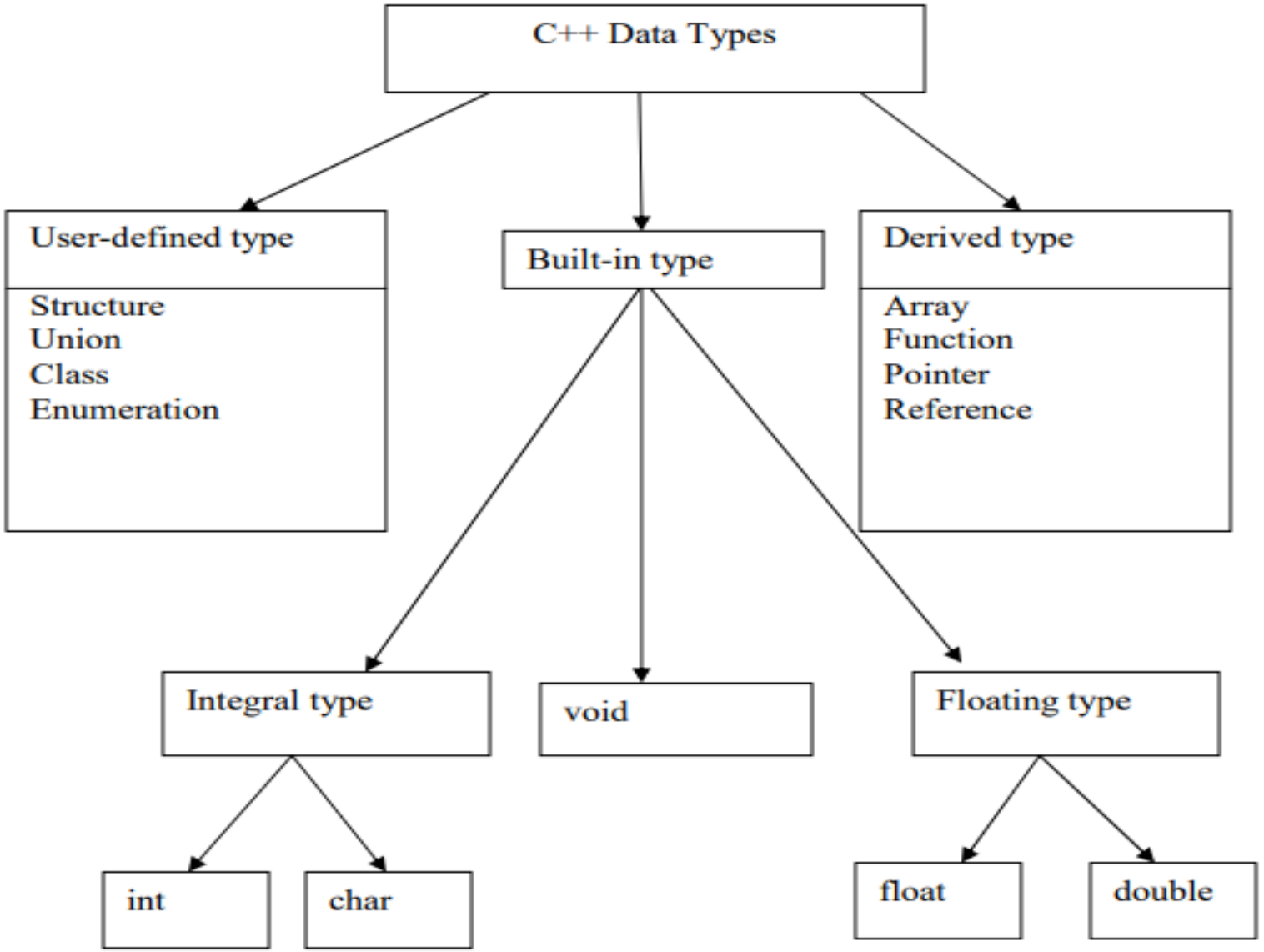
```
int myNum = 15;  // myNum is 15
myNum = 10;  // Now myNum is 10
cout << myNum;  // Outputs 10
```

```
int x = 5, y = 6, z = 50;
cout << x + y + z;
```

```
const int myNum = 15;  // myNum will always be 15
myNum = 10;  // error: assignment of read-only variable 'myNum'
```

C++ Data Types

A data type specifies the type of data that a variable can store such as integer, floating, character etc.



C++ Data Types

```
int myNum = 5;               // Integer (whole number)
float myFloatNum = 5.99;     // Floating point number
double myDoubleNum = 9.98;   // Floating point number
char myLetter = 'D';         // Character
bool myBoolean = true;       // Boolean
string myText = "Hello";     // String
```

Data Type	Size	Description
boolean	1 byte	Stores true or false values
char	1 byte	Stores a single character/letter/number, or ASCII values
int	2 or 4 bytes	Stores whole numbers, without decimals
float	4 bytes	Stores fractional numbers, containing one or more decimals. Sufficient for storing 6-7 decimal digits
double	8 bytes	Stores fractional numbers, containing one or more decimals. Sufficient for storing 15 decimal digits

Numeric Types

```
int myNum = 1000;
cout << myNum;
```

```
float myNum = 5.75;
cout << myNum;
```

```
double myNum = 19.99;
cout << myNum;
```

Boolean Types

A boolean data type is declared with the **bool** keyword and can only take the values true or **false**.

When the value is returned, **true = 1 and false = 0.**

```
bool isCodingFun = true;
bool isFishTasty = false;
cout << isCodingFun;  // Outputs 1 (true)
cout << isFishTasty;  // Outputs 0 (false)
```

Character & String Types

```
char a = 65, b = 66, c = 67;
cout << a;
cout << b;
cout << c;
```

Output: A B C

```
// Include the string library
#include <string>

// Create a string variable
string greeting = "Hello";

// Output string value
cout << greeting;
```

Character & String Types

```c
#include <stdio.h>

int main() {
    char ch;

    printf("Enter an uppercase character: ");
    scanf(" %c", &ch);
    if (ch >= 'A' && ch <= 'Z') {
        ch = ch + 32;
        printf("The lowercase character is: %c\n", ch);
    } else {
        printf("The entered character is not an uppercase letter.\n");
    }

    return 0;
}
```

Character & String Types

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    int i;
    printf("\nEnter a string : ");
    gets(s);

    for (i = 0; s[i]!='\0'; i++) {
        if(s[i] >= 'a' && s[i] <= 'z') {
            s[i] = s[i] - 32;
        }
    }
    printf("\nString in Upper Case = %s", s);
    return 0;
}
```

Character & String Types

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string s;

    cout << "Enter a string: ";
    getline(cin, s); // safe input


    for (size_t i = 0; i < s.length(); i++) {
        if (s[i] >= 'a' && s[i] <= 'z') {
            s[i] = s[i] - 32; // convert to uppercase
        }
    }

    cout << "String in Upper Case = " << s << endl;
    return 0;
}

```

Auto keyword

```
int main () {
  // Creating auto variables
  auto myNum = 5;           // int
  auto myFloatNum = 5.99;    // float
  auto myDoubleNum = 9.98;   // double
  auto myLetter = 'D';       // char
  auto myBoolean = true;     // bool
  auto myString = "Hello"; // std::string

  // Print variable values
  cout << "int: " << myNum << "\n";
  cout << "float: " << myFloatNum << "\n";
  cout << "double: " << myDoubleNum << "\n";
  cout << "char: " << myLetter << "\n";
  cout << "bool: " << myBoolean << "\n";
  cout << "string: " << myString << "\n";

  return 0;
}
```

Let's see the basic data types.

Data Types	Memory Size	Range
char	1 byte	-128 to 127
signed char	1 byte	-128 to 127
unsigned char	1 byte	0 to 127
short	2 byte	-32,768 to 32,767
signed short	2 byte	-32,768 to 32,767
unsigned short	2 byte	0 to 32,767
int	2 byte	-32,768 to 32,767
signed int	2 byte	-32,768 to 32,767
unsigned int	2 byte	0 to 32,767
short int	2 byte	-32,768 to 32,767
signed short int	2 byte	-32,768 to 32,767
unsigned short int	2 byte	0 to 32,767
long int	4 byte	
signed long int	4 byte	
unsigned long int	4 byte	
float	4 byte	
double	8 byte	
long double	10 byte	

C++ Keywords

A keyword is a reserved word. You cannot use it as a variable name, constant name etc. **A list of 32 Keywords in C++ Language which are also available in C language are given below.**

auto	break	case	char	const	continue	default	do
double	else	enum	extern	float	for	goto	if
int	long	register	return	short	signed	sizeof	static
struct	switch	typedef	union	unsigned	void	volatile	while

A list of 30 Keywords in C++ Language which are not available in C language are given below.

asm	dynamic_cast	namespace	reinterpret_cast	bool
explicit	new	static_cast	false	catch
operator	template	friend	private	class
this	inline	public	throw	const_cast
delete	mutable	protected	true	try
typeid	typename	using	virtual	wchar_t

C++ Identifiers

C++ identifiers in a program are used to refer to the name of the variables, functions, arrays, or other user-defined data types created by the programmer. They are the basic requirement of any language. Every language has its own rules for naming the identifiers.

In short, we can say that the C++ identifiers represent the essential elements in a program which are given below:

- **Constants**
- **Variables**
- **Functions**
- **Labels**
- **Defined data types**

Some naming rules are common in both C and C++. They are as follows:

•Only alphabetic characters, digits, and underscores are allowed.

•The identifier name cannot start with a digit, i.e., the first letter should be alphabetical. After the first letter, we can use letters, digits, or underscores.

•In C++, uppercase and lowercase letters are distinct. Therefore, we can say that C++ identifiers are case-sensitive.

•A declared keyword cannot be used as a variable name.

For example, suppose we have two identifiers, named as 'FirstName', and 'Firstname'. Both the identifiers will be different as the letter 'N' in the first case in uppercase while lowercase in second. Therefore, it proves that identifiers are case-sensitive.

Valid Identifiers

The following are the examples of valid identifiers are:

Result
Test2
_sum
power

Invalid Identifiers

The following are the examples of invalid identifiers:

```
Sum-1   // containing special character '-'.
2data   // the first letter is a digit.
break   // use of a keyword.
```

The following is the list of differences between identifiers and keywords:

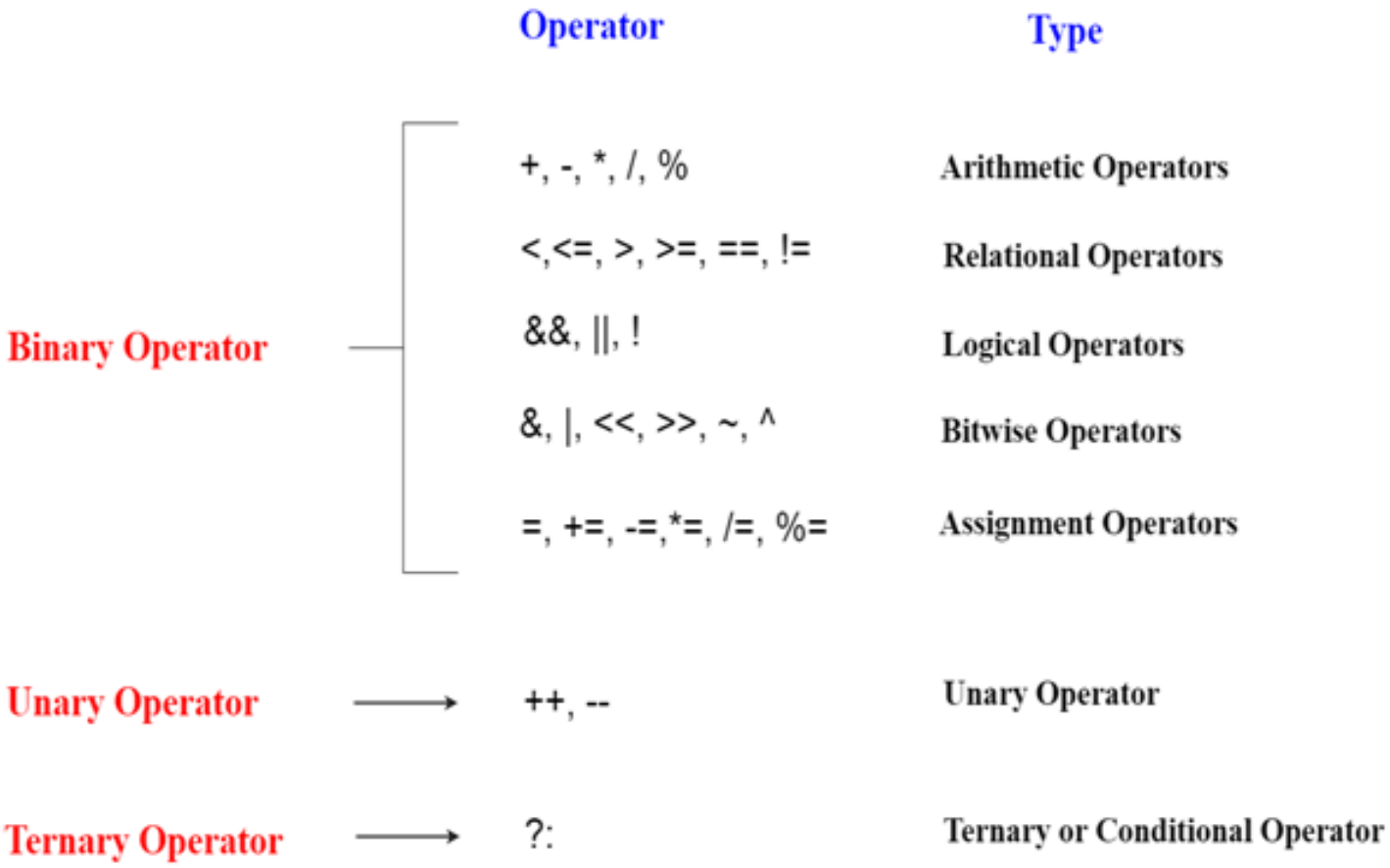
Identifiers	Keywords
Identifiers are the names defined by the programmer to the basic elements of a program.	Keywords are the reserved words whose meaning is known by the compiler.
It is used to identify the name of the variable.	It is used to specify the type of entity.
It can consist of letters, digits, and underscore.	It contains only letters.
It can use both lowercase and uppercase letters.	It uses only lowercase letters.
No special character can be used except the underscore and dollar ($) sign.	It cannot contain any special character.
The starting letter of identifiers can be lowercase, uppercase or underscore.	It can be started only with the lowercase letter.
It can be classified as internal and external identifiers.	It cannot be further classified.
Examples are test, result, sum, power, etc.	Examples are 'for', 'if', 'else', 'break', etc.

C++ Operators

An operator is simply a symbol that is used to perform operations. There can be many types of operations like arithmetic, logical, bitwise etc.

There are following types of operators to perform different types of operations in C language.

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Bitwise Operators
- Assignment Operator
- Unary operator
- Ternary or Conditional Operator
- Misc Operator



Arithmetic Operators

Operator	Name	Description	Example
+	Addition	Adds together two values	x + y
-	Subtraction	Subtracts one value from another	x - y
*	Multiplication	Multiplies two values	x * y
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	x % y
++	Increment	Increases the value of a variable by 1	++x
--	Decrement	Decreases the value of a variable by 1	--x

Assignment Operators

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
&=	x &= 3	x = x & 3
|=	x |= 3	x = x | 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

Relational/ Comparison Operators

Operator	Name	Example	Description
==	Equal to	x == y	Returns 1 if the values are equal
!=	Not equal	x != y	Returns 1 if the values are not equal
>	Greater than	x > y	Returns 1 if the first value is greater than the second value
<	Less than	x < y	Returns 1 if the first value is less than the second value
>=	Greater than or equal to	x >= y	Returns 1 if the first value is greater than, or equal to, the second value
<=	Less than or equal to	x <= y	Returns 1 if the first value is less than, or equal to, the second value

Logical Operators

Operator	Name	Description	Example
&&	Logical and	Returns true if both statements are true	x < 5 && x < 10
||	Logical or	Returns true if one of the statements is true	x < 5 || x < 4
!	Logical not	Reverse the result, returns false if the result is true	!(x < 5 && x < 10)

Bitwise Operators

Operator	Description	Example
&	Binary AND Operator copies a bit to the result if it exists in both operands.	(A & B)
|	Binary OR Operator copies a bit if it exists in either operand.	(A | B)
^	Binary XOR Operator copies the bit if it is set in one operand but not both.	(A ^ B)
~	Binary One's Complement Operator is unary and has the effect of 'flipping' bits.	(~A
<<	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	A << 2
>>	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	A >> 2

Ternary Operator

variable = Expression1 ? Expression2 : Expression3;

```
#include <stdio.h>

int main() {
int a = 10;
int b = 20;
int max;

max = (a > b) ? a : b; // If a > b is true, max = a; otherwise, max = b.

printf("The maximum value is: %d\n", max);

return 0;
}
```

C++ Strings

Strings are used for storing text/characters.

For example, "Hello World" is a string.

A string variable contains a collection of characters surrounded by double quotes:

```
#include <iostream>
#include <string>
using namespace std;

int main() {
  string greeting = "Hello";
  cout << greeting;
  return 0;
}
```

String Concatenation

```
string firstName = "John ";
string lastName = "Doe";
string fullName = firstName + lastName;
cout << fullName;
```

```
string firstName = "John";
string lastName = "Doe";
string fullName = firstName + " " + lastName;
cout << fullName;
```

```
string firstName = "John ";
string lastName = "Doe";
string fullName
= firstName.append(lastName);
cout << fullName;
```

C++ Numbers and Strings

If you add two numbers, the result will be a number:

```
int x = 10;
int y = 20;
int z = x + y;      // z will be 30 (an integer)
```

If you add two strings, the result will be a string concatenation:

```
string x = "10";
string y = "20";
string z = x + y;   // z will be 1020 (a string)
```

If you try to add a number to a string, an error occurs:

```
string x = "10";
int y = 20;
string z = x + y;
```

String Length

To get the length of a string, use the length() function:

```
string txt = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
cout << "The length of the txt string is: " << txt.length();
```

You might see some C++ programs that use the size() function to get the length of a string. This is just an alias of length(). It is completely up to you if you want to use length() or size():

```
string txt = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
cout << "The length of the txt string is: " << txt.size();
```

Special Characters

Escape character	Result	Description
\'	'	Single quote
\"	"	Double quote
\\	\	Backslash

Escape Character	Result
\n	New Line
\t	Tab

Special Characters

Because strings must be written within quotes, C++ will misunderstand this string, and generate an error:

The solution to avoid this problem, is to use the backslash escape character.

The **backslash (\) escape** character turns special characters into string character.

string txt = "We are the so-called "Vikings" from the north."; // error

The sequence **\"** inserts a double quote in a string:

string txt = "We are the so-called \"Vikings\" from the north.";

Output: We are the so-called "Vikings" from the north.

The sequence \' inserts a single quote in a string:

string txt = "It\'s alright.";

Output: It's alright.

Special Characters

The sequence \\ inserts a single backslash in a string:

string txt = "The character \\ is called backslash.";

Output: The character \ is called backslash.

string txt = "Hello\nWorld!";

Output: Hello
World!

string txt = "Hello\tWorld!";

Output: Hello World!

Math()

The max(x,y) function can be used to find the highest value of x and y:

And the min(x,y) function can be used to find the lowest value of x and y:

```
cout << max(5, 10);

cout << min(5, 10);

cout << sqrt(64);
cout << round(2.6);

cout << floor(2.6);
cout << ceil(5.6);

cout << abs(-664);
```

Precedence of Operators in C++

The precedence of operator species that which operator will be evaluated first and next. The associativity specifies the operators direction to be evaluated, it may be left to right or right to left.

Let's understand the precedence by the example given below:

```
int data=5+10*10;
```

The "data" variable will contain 105 because * (multiplicative operator) is evaluated before + (additive operator).

The precedence and associativity of C++ operators is given below:

Category	Operator	Associativity
Postfix	() [] -> . ++ - -	Left to right
Unary	+ - ! ~ ++ - - (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Right to left
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=/td>	Right to left
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR	|	Right to left
Logical AND	&&	Left to right
Logical OR	||	Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %=>>= <<= &= ^= |=	Right to left
Comma	,	Left to right

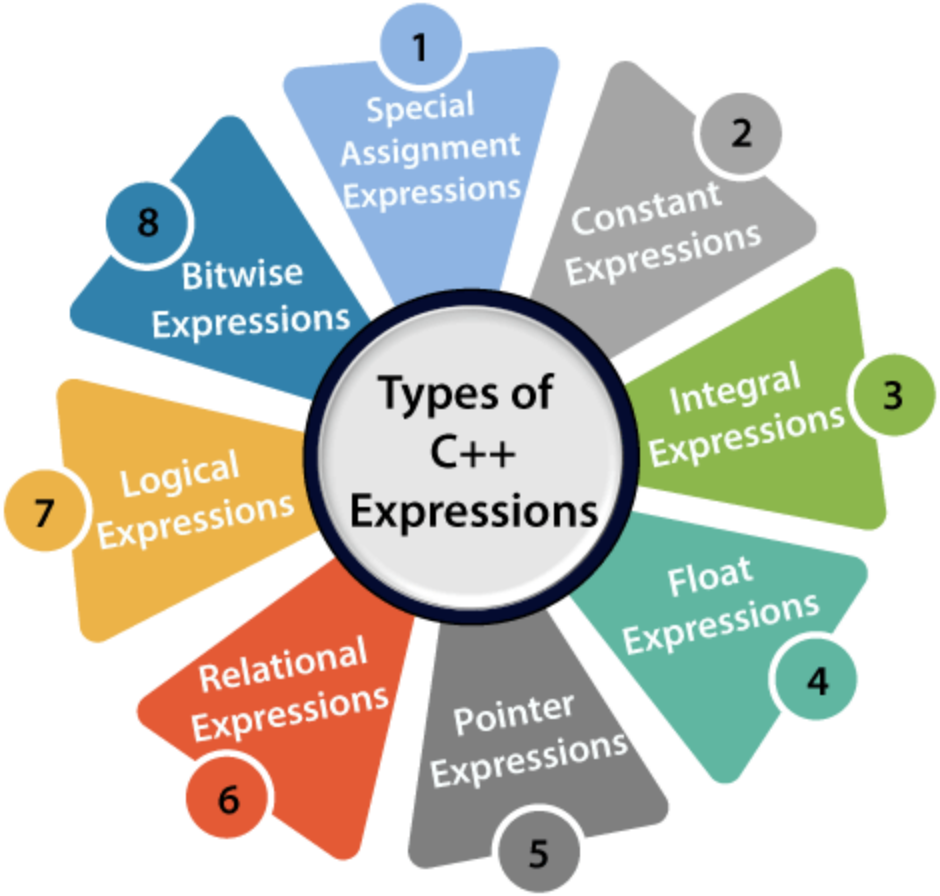
C++ Expression

C++ expression consists of operators, constants, and variables which are arranged according to the rules of the language. It can also contain function calls which return values. An expression can consist of one or more operands, zero or more operators to compute a value. Every expression produces some value which is assigned to the variable with the help of an assignment operator.

Examples of C++ expression:

1.(a+b) - c
2.(x/y) -z
3.4a2 - 5b +c
4.(a+b) * (x+y)

An expression can be of following types:



Constant expressions

A constant expression is an expression that consists of only constant values. It is an expression whose value is determined at the compile-time but evaluated at the run-time. It can be composed of integer, character, floating-point, and enumeration constants.

Constants are used in the following situations:

- It is used in the subscript declarator to describe the array bound.
- It is used after the case keyword in the switch statement.
- It is used as a numeric value in an **enum**
- It specifies a bit-field width.
- It is used in the pre-processor **#if**

In the above scenarios, the constant expression can have integer, character, and enumeration constants. We can use the static and extern keyword with the constants to define the function-scope.

The following table shows the expression containing constant value:

Expression containing constant	Constant value
x = (2/3) * 4	(2/3) * 4
extern int y = 67	67
int z = 43	43
static int a = 56	56

Integral Expressions

An integer expression is an expression that produces the integer value as output after performing all the explicit and implicit conversions.

Following are the examples of integral expression:

1. (x * y) -5
2. x + **int**(9.0)

where x and y are the integers.

Float Expressions

A float expression is an expression that produces floating-point value as output after performing all the explicit and implicit conversions.

The following are the examples of float expressions:

1) x+y
2) (x/10) + y
3) 34.5
4) x+**float**(10)

Pointer Expressions

A pointer expression is an expression that produces address value as an output.

The following are the examples of pointer expression:

1) &x
2) ptr
3) ptr++
4) ptr-

Relational Expressions

A relational expression is an expression that produces a value of type bool, which can be either true or false. It is also known as a boolean expression. When arithmetic expressions are used on both sides of the relational operator, arithmetic expressions are evaluated first, and then their results are compared.

The following are the examples of the relational expression:

1.a>b
2.a-b >= x-y
3.a+b>80

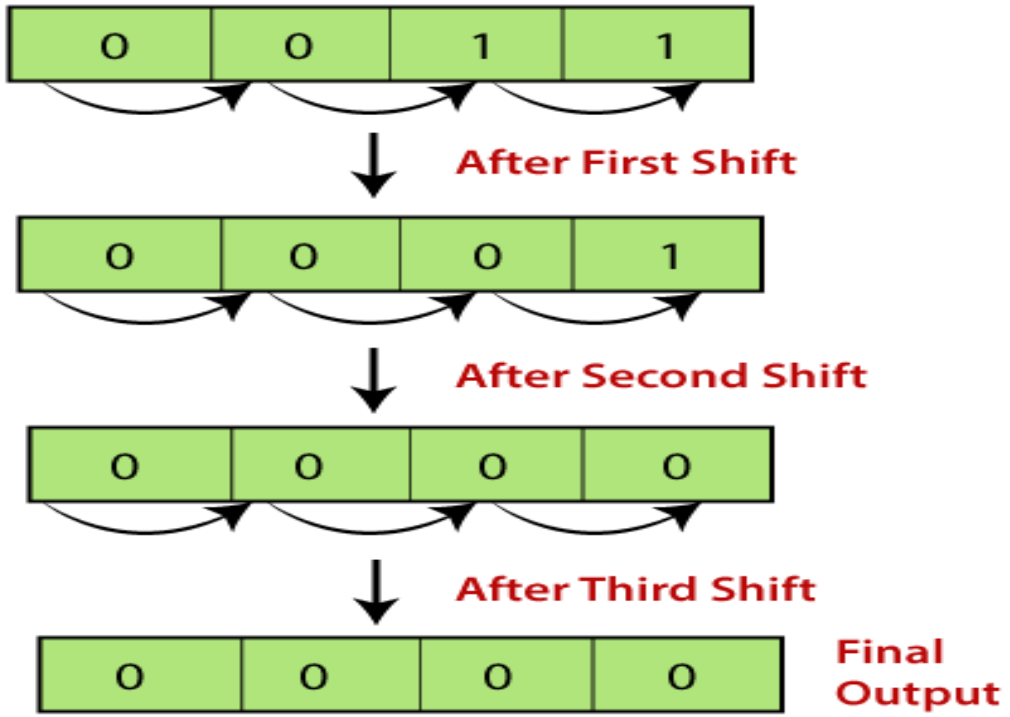
Logical Expressions

A logical expression is an expression that combines two or more relational expressions and produces a bool type value. The logical operators are '&&' and '||' that combines two or more relational expressions.

The following are some examples of logical expressions:
1.a>b && x>y
2.a>10 || b==5

Bitwise Expressions

A bitwise expression is an expression which is used to manipulate the data at a bit level. They are basically used to shift the bits.
For example:
x=3
x>>3 // This statement means that we are shifting the three-bit position to the right.
In the above example, the value of 'x' is 3 and its binary value is 0011. We are shifting the value of 'x' by three-bit position to the right. Let's understand through the diagrammatic representation.



Special Assignment Expressions

Special assignment expressions are the expressions which can be further classified depending upon the value assigned to the variable.

•Chained Assignment

Chained assignment expression is an expression in which the same value is assigned to more than one variable by using single statement.

For example:

a=b=20

or

(a=b) = 20

•Embedded Assignment Expression

An embedded assignment expression is an assignment expression in which assignment expression is enclosed within another assignment expression.

```
a=10+(b=90);  // embedded assignment expression
```

•Compound Assignment

A compound assignment expression is an expression which is a combination of an assignment operator and binary operator.

For example,

```
a+=10;
```